IMDB Spoiler Detection with Classical, Hybrid, Sequential, and Transformer-Based Models

Aylin Doğan
221101058
Computer Engineering Department
TOBB ETU
Ankara, Turkey
aylindogan@etu.edu.tr

Saadet Cansu Baktıroğlu
231401023
Artificial Intelligence Engineering Department
TOBB ETU
Ankara, Turkey
sbaktiroglu@etu.edu.tr

Emir Yücedağ
221101077
Computer Engineering Department
TOBB ETU
Ankara, Turkey
eyucedag@etu.edu.tr

***Abstract*—Online movie platforms host large volumes of user-generated reviews, many of which unintentionally reveal critical plot details (spoilers) that negatively impact user experience. Automatic spoiler detection is a challenging binary text classification problem due to implicit language use, long narrative structure, and significant class imbalance. In this study, we investigate spoiler detection on the IMDB Spoiler Dataset by systematically comparing classical, hybrid, sequential, and transformer-based machine learning approaches under a unified evaluation protocol. We evaluate five models: TF-IDF–based Logistic Regression and Linear SVM as classical baselines, a hybrid LightGBM model combining textual and metadata features, a sequential LSTM model capturing narrative flow, and a transformer-based DistilBERT model leveraging deep contextual representations. To prevent data leakage, all models are trained and evaluated using a movie-level grouped train/validation/test split. Performance is assessed using confusion matrices, F1-score, and ROC-AUC, with particular emphasis on spoiler recall and balanced performance. Experimental results show that while classical baselines remain competitive and computationally efficient, deep learning models significantly improve spoiler detection. LSTM outperforms bag-of-words approaches by modeling word order, and DistilBERT achieves the best overall performance, yielding the highest F1-score and lowest spoiler miss rate. The findings highlight the importance of contextual language modeling for implicit spoiler detection and demonstrate practical trade-offs between accuracy and computational cost.**

***Index Terms*—spoiler detection, IMDB reviews, TF-IDF, LightGBM, LSTM, DistilBERT, transformer**

I. Introduction – Problem Definition

Online movie platforms such as IMDB host millions of user-generated reviews that strongly influence viewing decisions. While these reviews provide valuable opinions and insights, they frequently contain spoilers, i.e., unintended revelations of critical plot elements. Exposure to spoilers can significantly reduce user enjoyment and trust in the platform. Relying on manual spoiler tagging by users is unreliable and does not scale to large datasets. Therefore, there is a strong practical need for automatic spoiler detection systems that can reliably identify spoiler content without human intervention.

In this project, spoiler detection is formulated as a binary text classification problem. Each movie review is classified into one of two categories: Spoiler (1): The review reveals key plot details.

Non-spoiler (0): The review does not reveal critical plot information.

The task is particularly challenging because spoiler cues are often implicit, contextual, and narrative-dependent, rather than based on explicit keywords. Additionally, the dataset exhibits class imbalance, with spoiler reviews forming a minority class.

The main purpose of this project is to design, implement, and evaluate multiple machine learning approaches for automatic spoiler detection, with the primary objective of detecting as many spoiler reviews as possible. From a user-experience perspective, the most critical requirement is that spoiler content should not be shown to users under any circumstances. Therefore, the project prioritizes maximizing spoiler detection (high recall), even at the cost of occasionally flagging non-spoiler reviews. The specific objectives of the project are as follows:

- To analyze the characteristics of spoiler and non-spoiler reviews through exploratory data analysis.
- To establish strong classical baselines using TF-IDF-based models such as Logistic Regression and Linear SVM.
- To evaluate the effectiveness of a gradient boosting approach (LightGBM) in leveraging structured and derived features (e.g., review length and metadata) for spoiler detection under class imbalance.
- To investigate whether sequential models (LSTM) improve spoiler detection by capturing word order and narrative flow in long reviews.
- To assess the effectiveness of transformer-based models (DistilBERT) in capturing deeper contextual semantics, particularly for implicit and long-range spoiler cues.
- To compare all models in terms of spoiler recall, F1-score, interpretability, and computational efficiency, under realistic deployment constraints where preventing spoiler exposure is prioritized over minimizing false positives.

Performance Metrics

Due to the imbalanced nature of the dataset and the high cost of missing spoilers, evaluation focuses on metrics that reflect balanced performance on the spoiler class. F1-score (particularly Macro-F1 and spoiler-class F1) is used as the primary

metric, as it jointly captures precision and recall at a fixed operating point and directly reflects the trade-off between false positives and false negatives. Precision–Recall Area Under the Curve (PR-AUC) is reported as a complementary metric because it evaluates model performance across all possible decision thresholds and is especially informative in imbalanced classification settings, where the positive class (spoilers) is rare. PR-AUC reflects how well a model ranks spoiler reviews ahead of non-spoiler reviews, independent of a single threshold choice. ROC-AUC is also included to measure overall class separability based on prediction scores. While ROC-AUC is less sensitive to class imbalance than PR-AUC, it provides a useful global view of a model's discriminative ability and allows comparison with prior studies that commonly report ROC-AUC. Overall accuracy is included for completeness but is not treated as a primary indicator of success, as it can be misleading in imbalanced datasets where correctly predicting the majority class yields high accuracy without effective spoiler detection

II. Literature Review

Spoiler detection research has evolved from manual feature engineering to deep contextual models. The development of this field can be summarized through these key milestones:

- **Classical Baselines:** Early research utilized classical machine learning, notably SVMs (Ishihara & Sekine), to identify lexical and surface-level features in short texts.
- **The IMDB Benchmark:** A major turning point was the release of the IMDB Spoiler Dataset by Misra. This work identified critical challenges such as significant class imbalance and the correlation between review length and spoiler probability, forming the foundation for our study.
- **Genre and Context:** Chang et al. demonstrated that spoiler patterns are genre-dependent (e.g., Mystery vs. Action). Our exploratory analysis aligns with these findings, observing higher spoiler frequencies in plot-driven genres like Horror and Science Fiction.
- **Deep Learning & Transformers:** Recent shifts favor neural architectures. Pielka et al. showed that fine-tuned Transformers outperform TF-IDF baselines, while models like GUSD and MVSD integrated metadata and graph-based structures to capture complex relationships.

Our Contribution: Unlike prior studies focusing on single model families, this work provides a unified comparison of classical models, sequential networks (LSTM), and Transformers (DistilBERT). We emphasize performance under realistic constraints, prioritizing F_1-score, recall, and computational efficiency.

III. Dataset, Data Characteristics, and Features

A. Data Source and Dataset Description

The dataset used in this study is the IMDB Spoiler Dataset, which is publicly available on Kaggle.[1] It consists of 573,913 user-generated movie reviews collected from 1,572 distinct movies. Each review is annotated with a binary label indicating whether it contains a spoiler (`is_spoiler = 1`) or not (`is_spoiler = 0`).

The dataset includes both unstructured textual data and structured metadata, making it suitable for evaluating a wide range of machine learning models. The primary data files are:

- `IMDB_reviews.json`: review-level data containing `movie_id`, `review_text`, and `is_spoiler`.
- `IMDB_movie_details.json`: movie-level metadata including `movie_id`, genre, rating, and release date.

Approximately 26–26.5% of the reviews are labeled as spoilers, while the remaining 73–74% are non-spoilers, resulting in a moderately imbalanced binary classification problem. This imbalance is a central consideration in both model design and evaluation.

B. Data Characteristics and Feature Types

The dataset contains the following types of features:

- **Plain text data:**
  - `review_text`: free-form user reviews (unstructured, variable length).
- **Binary data:**
  - `is_spoiler` (0 = non-spoiler, 1 = spoiler).
- **Numerical features:**
  - Movie rating.
  - Review-derived features such as word count and character count.
- **Categorical / nominal data:**
  - Movie genres (multi-label, unordered; e.g., Drama, Thriller, Horror).
- **Temporal data:**
  - Movie release date, used for temporal and correlation analysis.

The core prediction task is binary classification, where the objective is to detect spoiler reviews with high reliability.

C. Data Cleaning and Preprocessing

Although the IMDB Spoiler Dataset is well-structured, several preprocessing steps are required to ensure consistency and reliability across models.

1) Handling Missing and Erroneous Data: Reviews with missing values in `review_text` or `is_spoiler` are removed. Reviews whose text becomes empty after whitespace stripping are also discarded. Missing textual entries are not imputed, as empty reviews provide no semantic value. After cleaning, all remaining samples contain valid text and labels.

2) Text Cleaning: A lightweight, rule-based preprocessing pipeline is applied uniformly across models:

- Lowercasing all characters to reduce vocabulary sparsity.
- Removal of HTML tags (e.g., `<br />`) using regular expressions.
- Removal of URLs (e.g., strings starting with `http` or `www`).

[1] https://www.kaggle.com/datasets/rmisra/imdb-spoiler-dataset

- Whitespace normalization by collapsing multiple spaces and line breaks.

These steps preserve semantic content while reducing noise and inconsistencies in raw user text.

3) Length-Based Filtering and Derived Features: Exploratory analysis shows that extremely short reviews are typically uninformative. Therefore, reviews with five or fewer tokens are removed.

Several derived numerical features are computed:

- Word count.
- Character count.
- Token length.

These features are later used both for analysis and for hybrid models (e.g., LightGBM).

D. Metadata Integration and Feature Engineering

To enrich the dataset, review-level data are merged with movie-level metadata using `movie_id` as a key. Movie genres are converted into Python lists and encoded using multi-label binarization (e.g., `Genre_Action`, `Genre_Thriller`). Movie rating and release date are retained for correlation and temporal analysis.

This integration enables investigation of genre-specific spoiler behavior, as suggested by prior work and confirmed by our exploratory data analysis.

E. Exploratory Data Analysis Findings

Exploratory data analysis reveals several important patterns:

- **Class imbalance:** Spoilers form a minority class (approximately 26%), motivating recall-oriented evaluation.
- **Review length correlation:** Spoiler reviews are significantly longer and more detailed than non-spoiler reviews.
- **Genre effects:** Higher spoiler rates are observed in plot-driven genres such as Mystery, Horror, Thriller, and Science Fiction.
- **Weak rating correlation:** Movie ratings show only a weak relationship with spoiler occurrence.

These findings motivate the inclusion of both textual and structural features in downstream models.

IV. Models Used

A. Task Setting and Splits

This project addresses a binary text classification task, where each review is labeled as `is_spoiler` $\in \{0, 1\}$. Due to the risk of content leakage—arising from reviews of the same movie sharing plot-specific vocabulary—we adopt a movie-level grouped split using `movie_id`.

The dataset is divided into Train / Validation / Test splits of 70% / 15% / 15%, ensuring that reviews from the same movie do not appear in multiple splits. This setup simulates a realistic deployment scenario in which the model encounters reviews for previously unseen movies.

To verify stability, we additionally apply 5-fold stratified cross-validation on the training data for classical baseline models. Low fold-to-fold variance indicates that the observed performance is not dependent on a favorable split.

B. Baseline Reference Points

Before training the final models, we establish simple reference baselines:

- **Majority-class baseline (DummyClassifier):** This model always predicts the non-spoiler class. It achieves high accuracy close to the non-spoiler rate, but exhibits very weak minority-class performance and a PR-AUC close to the spoiler prior, illustrating that accuracy alone is misleading in imbalanced settings.
- **Default linear baselines (Logistic Regression / Linear SVM):** These models are trained on the same TF–IDF feature space using default hyperparameters. They provide stronger reference points than the majority baseline by capturing term importance, but remain limited compared to tuned models and threshold-optimized operating points.

These baselines are used to quantify the gains achieved through hyperparameter tuning and decision-threshold calibration.

C. Shared Text Representation for Classical Models: TF–IDF

Both Logistic Regression and Linear SVM are trained on a shared TF–IDF feature representation constructed from review text:

- Word-level TF–IDF vectors.
- N-gram range: unigram versus unigram + bigrams (typically $(1, 2)$).
- Controlled vocabulary size via `max_features` (e.g., 5k, 10k, 20k, 50k).
- Appropriate `min_df` and `max_df` thresholds.
- Stop-word removal using `stop_words="english"` in the main experiments.

Using a shared representation ensures that observed performance differences reflect classifier behavior rather than discrepancies in feature extraction. Empirically, incorporating bigrams improves performance, indicating that short phrases (e.g., “in the end”, “turns out”, “the killer”) carry spoiler-relevant signals beyond single words.

D. Model 1: TF–IDF + Logistic Regression

1) Methodology and Motivation: Logistic Regression is employed as a strong classical baseline for high-dimensional, sparse text representations. In the IMDB spoiler detection setting, it offers three key advantages:

- It scales efficiently to hundreds of thousands of reviews on CPU.
- It handles TF–IDF feature spaces effectively.
- It produces probabilistic outputs for the spoiler class, enabling explicit decision-threshold calibration for recall-oriented spoiler filtering.

2) Hyperparameter Exploration: With the TF–IDF configuration fixed (n-gram range, `max_features`, `min_df`, `max_df`, and stop-word handling), hyperparameter tuning for Logistic Regression focuses on:

- Regularization strength C.

- Penalty type: L1 versus L2.
- Solver choice: `liblinear`, `lbfgs`, and `saga`.
- Decision threshold on predicted spoiler probabilities.

3) Key Observations: For very small values of C (e.g., $C = 0.01$), the model is strongly regularized and underfits:

- The decision boundary is overly conservative.
- Spoiler recall remains only moderate.
- Precision is relatively low, leading to many false positives despite acceptable overall accuracy.

For $C \geq 0.1$, accuracy stabilizes in the low-0.70 range. Spoiler precision increases slightly, while recall decreases only marginally. In this regime, both macro and weighted F1-scores become stable, and performance is relatively insensitive to further increases in C. Consequently, a moderate regularization strength such as $C = 1$ (or similarly $C \approx 0.1$) is preferred, as it provides a robust trade-off between model complexity, accuracy, and spoiler-class recall and precision.

Comparing penalty types, L2 regularization yields slightly better separability for the spoiler class—reflected in higher F1 and ROC-AUC scores—than L1 regularization. While L1 produces a sparser and more interpretable model, it does so at the cost of a small reduction in spoiler recall and F1-score. Therefore, L2 is selected as the default configuration, with L1 reserved for scenarios where coefficient sparsity is critical.

4) Threshold Tuning (Recall-Oriented): Because Logistic Regression outputs calibrated probabilities for the spoiler class, the decision threshold can be tuned explicitly for different operating regimes. Thresholds below the default value of 0.5 are evaluated to increase spoiler recall (i.e., reduce false negatives) at the expense of additional false positives. Empirically:

- Lowering the threshold monotonically increases recall while reducing precision.
- A threshold around 0.45 emerges as a practical operating point, achieving high spoiler recall (approximately 0.73 in our experiments) while keeping overall accuracy and F1-score within acceptable bounds.

This property makes Logistic Regression particularly well-suited for recall-prioritized spoiler filtering, where missing spoilers is more costly than occasionally over-flagging non-spoiler reviews.

E. Model 2: TF–IDF + Linear SVM

1) Methodology and Motivation: The second linear model is a margin-based classifier: a Linear Support Vector Machine (LinearSVC) trained on the same TF–IDF feature representation. Linear SVMs are well known to perform strongly on sparse, high-dimensional text vectors and to scale efficiently on standard hardware.

Unlike Logistic Regression, LinearSVC does not produce calibrated probabilities. Instead, it outputs decision scores corresponding to signed distances from the separating hyperplane. Although these scores are not probabilities, they still enable thresholding and operating-point control via the `decision_function`.

2) Hyperparameter Exploration: The following hyperparameters are explored:

TF–IDF-related:

- N-gram range: unigrams versus unigrams + bigrams.
- Vocabulary size via `max_features` (e.g., 5k, 10k, 20k, 50k).

SVM-related:

- Regularization strength C.
- Loss function: `hinge` versus `squared_hinge`.
- Maximum number of iterations (`max_iter`) for convergence control.
- Decision threshold applied to `decision_function` scores.

3) Key Observations: **N-gram range and vocabulary size.** Moving from unigram-only features to a combination of unigrams and bigrams consistently improves accuracy and spoiler-class metrics. This confirms that short phrase patterns (e.g., “in the end”, “turns out”, “the killer”) provide important spoiler cues beyond individual words.

Increasing `max_features` from 5k to 10k yields a modest improvement in spoiler precision while maintaining recall in the mid-to-upper 0.6 range. However, further expansion to 20k or 50k features increases training cost and model size without meaningful performance gains. In some cases, test precision and accuracy even degrade slightly. This “vocabulary inflation” effect indicates that very large vocabularies are not cost-effective for this task.

Regularization strength. Scanning moderate values of C (e.g., $C = 1, 10, 100$) produces only minor variations in recall and F1-score, suggesting that the primary performance bottleneck lies in the linear TF–IDF representation rather than fine-grained tuning of C. When C is increased to very large values ($C \geq 1000$), both training and validation performance degrade, indicating numerical or optimization issues rather than improved margin control. Consequently, moderate C values are adopted in the final configuration, consistent with the Logistic Regression setup.

Loss function and convergence. With TF–IDF parameters fixed, increasing `max_iter` from 2,000 to 10,000 has negligible effect, indicating early convergence. Switching from `hinge` to `squared_hinge` yields a small but consistent improvement in accuracy and precision while leaving recall largely unchanged. Therefore, `squared_hinge` is selected as the preferred loss function.

4) Threshold Tuning Using Decision Scores: Although LinearSVC does not output probabilities, its decision scores can be thresholded to control the recall–precision trade-off. Around the default threshold (approximately 0), the model achieves balanced but modest performance, with spoiler recall in the 0.66–0.67 range and precision around 0.46–0.47.

Shifting the decision threshold toward more aggressive positive predictions substantially increases spoiler recall:

- Moderate threshold shifts increase recall to approximately 0.80, with a corresponding reduction in precision.

- More aggressive shifts can exceed 0.90 recall, but at the cost of severe precision and overall accuracy degradation due to a large number of false positives.

These results confirm that, as with Logistic Regression, operating-point selection via thresholding is the most effective control mechanism for recall-prioritized spoiler filtering.

5) Cross-Validation Stability: Using 5-fold stratified cross-validation, spoiler-class F1-scores for the Linear SVM model exhibit very low variance across folds. This indicates that performance is not driven by a favorable train–validation split and that the model generalizes consistently across different partitions of the training data. Similar stability is observed for Logistic Regression, reinforcing that performance differences between models reflect intrinsic modeling characteristics rather than data-splitting artifacts.

F. Model 3: Hybrid Gradient Boosting Approach (LightGBM)

1) Task Definition and Model Selection: The spoiler detection task is formulated as a binary classification problem, where the objective is to predict whether a given review belongs to the spoiler or non-spoiler class. For this task, we employ the Light Gradient Boosting Machine (LightGBM) [1], an efficient implementation of gradient-boosted decision trees.

The motivation for selecting LightGBM is threefold:

- **Non-linearity:** Unlike linear baselines, gradient-boosted trees naturally capture non-linear decision boundaries and complex feature interactions.
- **Hybrid capability:** LightGBM enables direct integration of heterogeneous feature types, combining high-dimensional text representations with structured metadata.
- **Imbalance handling:** The framework provides native mechanisms to address class imbalance, which is critical given that spoilers constitute approximately 26% of the dataset.

2) Data Splitting Strategy: To prevent data leakage and ensure robust evaluation, we avoid random splitting and instead apply a grouped splitting strategy using `movie_id`. Specifically, `GroupShuffleSplit` is used to partition the data into Training (70%), Validation (15%), and Test (15%) sets. This ensures that reviews from the same movie do not appear in multiple splits, closely simulating real-world deployment conditions.

3) Hybrid Feature Representation: The LightGBM model is trained on a hybrid feature set informed by exploratory data analysis:

- **Textual features:** TF–IDF vectors derived from review text using a limited vocabulary of the top 5,000 features and an n-gram range of unigrams and bigrams.
- **Structural numeric features:** Review-level features such as `word_count` and `char_count`, motivated by the observation that spoiler reviews tend to be more verbose.
- **Categorical metadata:** Movie genres encoded via multi-label binarization (e.g., `Genre_Action`, `Genre_Thriller`) to capture genre-specific spoiler tendencies.

4) Imbalance Handling and Hyperparameter Tuning: Given the class imbalance, `class_weight = balanced` is applied to penalize misclassification of the minority class. Hyperparameter optimization is conducted sequentially on the validation set:

- **Structural parameters:** `num_leaves` is set to 31, with `max_depth = -1` (unrestricted) to allow flexible tree growth.
- **Boosting iterations:** The number of trees is determined via early stopping, with training converging at approximately 736 boosting iterations, maximizing PR-AUC while preventing overfitting.
- **Regularization:** To mitigate noise from TF–IDF features, L1 and L2 regularization terms are tuned. A combination of `reg_alpha = 1.0` and `reg_lambda = 0.5` yields the most stable performance.

G. Model 4: Sequential Deep Learning Model (LSTM)

1) Motivation: Classical bag-of-words models such as TF–IDF ignore word order and treat documents as unordered collections of tokens. However, spoiler content in movie reviews often follows a narrative structure, in which critical information is revealed progressively (e.g., setup $\rightarrow$ development $\rightarrow$ reveal). This motivates the use of sequential models that can explicitly capture word order and local temporal dependencies.

2) Architecture and Training (Implementation-Aligned): The LSTM model employed in this study follows a standard sequence modeling architecture aligned with the actual implementation. It consists of:

- An embedding layer that maps discrete tokens to dense vector representations.
- One or more LSTM layers that model sequential dependencies in the review text.
- A linear output layer that produces logits for the two classes (spoiler vs. non-spoiler).

To address the class imbalance inherent in the dataset, training uses a weighted cross-entropy loss, where class weights are computed from the training labels. This ensures that misclassification of spoiler reviews is penalized more heavily than misclassification of non-spoiler reviews.

Optimization is performed using the Adam optimizer, which provides adaptive learning rates and stable convergence for deep neural networks. L2 weight regularization (weight decay) is applied during optimization to discourage overly large weights and improve generalization.

Additional stabilization and regularization techniques include:

- Dropout applied to LSTM outputs to reduce overfitting.
- Gradient clipping to prevent exploding gradients during backpropagation.
- Early stopping based on validation loss to avoid overtraining.
- Shuffling of training mini-batches at each epoch, applied only to the training split, while validation and test sets remain fixed to ensure consistent evaluation.

3) Limitations: Despite their ability to model word order and local narrative flow, LSTMs process tokens sequentially, which limits parallelization and makes capturing very long-range dependencies challenging for lengthy reviews. These limitations motivate the exploration of transformer-based architectures that rely on global attention mechanisms.

H. Model 5: Transformer-Based Model (DistilBERT / Tiny DistilBERT)

1) Motivation: Transformer-based models overcome the limitations of recurrent architectures by modeling global context through self-attention. In this framework, each token can attend to all other tokens in the sequence simultaneously. This property is particularly valuable for spoiler detection, where spoiler cues may appear far apart within a review and depend on relationships between distant phrases.

2) Model Choice and Practical Constraints: DistilBERT is a transformer model obtained via knowledge distillation from BERT, retaining most of BERT's representational power while significantly reducing model size and computational cost. However, in our experimental setup, fine-tuning the full DistilBERT model required approximately seven hours, which was impractical given the available time and hardware constraints.

For this reason, we employed Tiny DistilBERT, a lightweight variant of DistilBERT. Tiny DistilBERT substantially reduces the number of parameters and training time while preserving similar performance trends, particularly with respect to spoiler recall and Macro-F1 score. This choice reflects a realistic trade-off between model capacity and computational efficiency.

3) Fine-Tuning Setup: During fine-tuning, the following hyperparameters were explored:

- Learning rate.
- Batch size.
- Number of training epochs.
- Weight decay (L2 regularization).
- Class balancing strategies.

As with the LSTM model, training samples were shuffled at each epoch to improve stochastic gradient optimization and reduce batch-order effects. Validation and test sets were kept fixed to ensure fair and reproducible evaluation.

I. Why These Models Were Chosen: A Progressive Strategy

The overall modeling strategy follows a progressive refinement approach:

- **Logistic Regression / Linear SVM (TF–IDF):** strong, interpretable, and computationally inexpensive baselines.
- **LightGBM:** a non-linear hybrid model combining text features with metadata and derived structural features.
- **LSTM:** sequential modeling to capture narrative flow and local temporal dependencies.
- **DistilBERT:** attention-based contextual modeling for implicit and long-range spoiler cues.

This progression enables a systematic analysis of how increasing representational power influences spoiler detection performance under consistent evaluation and data-splitting constraints.

V. Test Results and Discussion

This section presents a comprehensive evaluation of the five models used in this study—Logistic Regression (LR), Linear SVM, Hybrid LightGBM, LSTM, and DistilBERT—based exclusively on results obtained from the held-out test set. Since the task involves binary spoiler detection under class imbalance, results are primarily interpreted through confusion matrices, F1-score, and ROC-AUC, with particular emphasis on spoiler-class recall and balanced performance.

A. Evaluation Setup and Metrics

All models are evaluated on the same movie-level grouped test split, ensuring that no reviews from the same movie appear in both training and test sets. This prevents data leakage and provides a realistic estimate of generalization performance.

Given the imbalanced nature of the dataset and the high cost of missing spoilers, the following evaluation metrics are emphasized:

- **Confusion matrix** (TP, TN, FP, FN) for detailed error analysis.
- **Recall (spoiler class)** to minimize missed spoilers.
- **F1-score** (macro / spoiler-class) as the primary comparison metric reflecting the precision–recall trade-off.
- **ROC-AUC** to assess ranking quality and class separability across decision thresholds.

Although ROC-AUC does not correspond to a single operating point, it provides a threshold-independent measure of model discrimination and complements recall- and F1-based analyses.

B. Model-wise Test Results and Interpretation

1) Logistic Regression (TF–IDF Baseline): **Confusion matrix analysis.** Logistic Regression achieves a high number of true negatives (TN = 58,948), indicating strong recognition of non-spoiler reviews. However, it misses a substantial number of spoilers (FN = 14,800). Lowering the decision threshold improves spoiler recall but introduces a large number of false positives.

ROC-AUC and F1. ROC-AUC ≈ 0.769, F1-score ≈ 0.55.

Interpretation. Logistic Regression effectively captures explicit lexical spoiler cues but struggles with implicit or contextual spoilers. Its main strengths are simplicity, interpretability, and computational efficiency, making it a solid baseline but insufficient for strong spoiler protection.

2) Linear SVM (TF–IDF Baseline): **Confusion matrix analysis.** The Linear SVM achieves moderate spoiler detection (TP = 10,232) with fewer false positives than Logistic Regression, but still misses a non-trivial number of spoilers (FN = 4,861). Spoiler recall is highly sensitive to the chosen decision threshold.

ROC-AUC and F1. ROC-AUC ≈ 0.768, F1-score ≈ 0.549.

TABLE I
TEST-SET PERFORMANCE COMPARISON ACROSS MODELS

Model	F1-score	ROC-AUC
DistilBERT	0.769	0.776
LSTM	0.742	0.756
LightGBM	0.546	0.767
Linear SVM	0.549	0.768
Logistic Regression	0.550	0.769

Interpretation. Linear SVM is a robust margin-based classifier for sparse text representations. While threshold tuning enables recall-oriented operation, the bag-of-words assumption limits its ability to detect implicit spoilers.

3) Hybrid LightGBM (TF–IDF + Metadata): **Confusion matrix analysis.** LightGBM demonstrates strong non-spoiler recognition (TN = 43,345) and improves spoiler recall relative to linear baselines, but still produces a notable number of false negatives (FN = 7,340). The model adopts a slightly more aggressive spoiler detection strategy, increasing false positives.

ROC-AUC and F1. ROC-AUC ≈ 0.767, F1-score ≈ 0.546.

Interpretation. The hybrid approach confirms the value of structural features such as review length and genre. However, despite non-linear modeling, TF–IDF-based text features lack the semantic depth required to capture subtle or implicit spoilers.

4) LSTM (Sequential Deep Learning Model): **Confusion matrix analysis.** The LSTM model substantially improves spoiler detection over classical models, achieving a high number of true positives (TP = 12,190) and fewer missed spoilers (FN = 10,448). Training curves exhibit some instability, reflecting the inherent challenges of sequential optimization.

ROC-AUC and F1. ROC-AUC ≈ 0.756, F1-score ≈ 0.742.

Interpretation. By modeling word order and narrative flow, LSTM captures spoiler structures such as buildup followed by reveal. However, its sequential nature limits the modeling of very long-range dependencies, explaining the remaining performance gap relative to transformer-based models.

5) DistilBERT (Transformer-Based Model): **Confusion matrix analysis.** DistilBERT achieves the highest true positive count and the lowest false negative rate among all evaluated models, indicating the strongest spoiler detection capability while maintaining reasonable control over false positives.

ROC-AUC and F1. ROC-AUC ≈ 0.776, F1-score ≈ 0.769.

Interpretation. DistilBERT's self-attention mechanism enables global, bidirectional context modeling, allowing detection of spoilers even when cues are implicit or widely separated in the text. This explains its consistent superiority across confusion matrices and ROC curves.

C. Cross-Model Comparison

Key observations include:

- DistilBERT clearly outperforms all other models in both F1-score and ROC-AUC.
- LSTM provides a strong middle ground, substantially improving over classical baselines.
- LightGBM benefits from metadata integration but lacks deep semantic understanding.
- Linear SVM and Logistic Regression remain efficient and stable baselines but are limited in detecting implicit spoilers.

D. Error Patterns and Practical Trade-offs

Across all models, false negatives (missed spoilers) are the most harmful errors, as they directly expose users to unwanted plot information. False positives are comparatively less damaging, as they merely over-warn users.

Classical models require aggressive threshold tuning to reduce false negatives, which sharply increases false positives. LightGBM offers a reasonable balance at moderate computational cost, while DistilBERT minimizes false negatives most effectively while maintaining balanced overall performance.

E. Final Discussion

The results demonstrate that spoiler detection strongly benefits from context-aware modeling. While classical TF–IDF baselines remain competitive and computationally efficient, deep learning—particularly transformer-based architectures—provides substantial gains in detecting implicit and narrative spoilers.

Under realistic deployment constraints, DistilBERT represents the best-performing solution, while LSTM and LightGBM offer meaningful trade-offs between performance and computational cost.

VI. CONCLUSIONS

In this study, we addressed the problem of automatic spoiler detection in movie reviews as a binary text classification task under class imbalance. Using the IMDB Spoiler Dataset, we systematically designed, implemented, and evaluated five modeling approaches: classical TF–IDF-based baselines (Logistic Regression and Linear SVM), a hybrid gradient boosting model (LightGBM), a sequential deep learning model (LSTM), and a transformer-based model (DistilBERT). All models were evaluated under a movie-level train/validation/test split to prevent data leakage and ensure realistic generalization.

A. Summary of Findings

Our experimental results demonstrate that:

- Classical models such as Logistic Regression and Linear SVM provide strong and computationally efficient baselines, effectively capturing explicit lexical spoiler cues.
- The hybrid LightGBM model benefits from integrating structural metadata such as review length and genre, confirming insights obtained during exploratory data analysis.
- The LSTM model improves spoiler detection by modeling word order and narrative flow, outperforming classical baselines in terms of balanced performance.
- DistilBERT consistently achieves the best results across F1-score, recall, and ROC-AUC, substantially reducing

missed spoilers by leveraging global, bidirectional context through self-attention.

Overall, transformer-based contextual modeling proves to be the most effective approach for detecting implicit and long-range spoiler cues.

B. Lessons Learned and Contributions

This study yields several important insights:

- Spoiler detection is inherently context-dependent, and models that capture deeper semantic information outperform bag-of-words approaches.
- Recall-oriented evaluation is critical, as missing spoilers is more harmful than over-flagging non-spoiler reviews.
- Even simple structural features, such as review length, serve as strong indicators of spoiler likelihood.
- Careful dataset splitting at the movie level is essential to avoid overly optimistic evaluation results caused by information leakage.

The main contribution of this work is a systematic and fair comparison of classical, hybrid, sequential, and transformer-based models under a unified evaluation protocol, highlighting practical trade-offs between performance, interpretability, and computational cost.

C. Limitations of the Study

Several limitations should be acknowledged:

- We did not apply extensive text-specific preprocessing or data augmentation, relying instead on relatively lightweight cleaning procedures.
- Due to computational constraints, full-scale fine-tuning of DistilBERT was not feasible; a Tiny DistilBERT variant was used instead.
- The study focused on review-level classification and did not explore sentence-level or token-level spoiler localization.
- Statistical significance testing across repeated deep learning runs was limited, as training transformer models multiple times is computationally expensive.

These factors may have constrained the achievable performance, particularly for deep learning models.

D. Future Work

Several promising directions can extend this research:

- Exploring full BERT or larger transformer models, as well as domain-adapted pretraining on movie-related corpora.
- Investigating sentence-level or span-level spoiler detection to enable partial masking rather than blocking entire reviews.
- Incorporating additional metadata, such as user behavior signals, temporal dynamics, or interaction graphs.
- Applying cost-sensitive learning or adaptive thresholding strategies to further optimize recall–precision trade-offs.
- Studying real-time deployment strategies that combine lightweight classical filters with transformer-based re-ranking.

In conclusion, this work confirms that modern contextual language models are highly effective for spoiler detection, while also emphasizing the importance of evaluation design and practical deployment considerations in real-world NLP systems.

APPENDIX

A. LightGBM Outcomes

1) ROC Curve: Figure **??** illustrates the ROC curve for the LightGBM model evaluated on the test set.

2) Confusion Matrix: Figure **??** presents the confusion matrix for LightGBM, highlighting the trade-off between spoiler recall and false positives.

3) Regularization Effects: Figure **??** shows the impact of L1 and L2 regularization on validation performance.

4) Log Loss and Overfitting Check: Figure **??** plots training and validation log loss across boosting iterations to assess overfitting behavior.

5) Learning Curve: Figure **??** depicts the learning curve of the LightGBM model over boosting rounds.

6) Feature Importance: Figure **??** shows the feature importance scores learned by LightGBM, highlighting the contribution of both textual and structural features.

REFERENCES

[1] G. Ke *et al.*, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[2] A. Vaswani *et al.*, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *NAACL-HLT*, 2019.

[4] V. Sanh *et al.*, "DistilBERT, a distilled version of BERT," *arXiv:1910.01108*, 2019.

[5] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[6] T. Joachims, "Text Categorization with Support Vector Machines," *ECML*, 1998.

[7] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[8] C. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.

[9] R. Misra, "IMDB Spoiler Dataset," 2019. [Online]. Available: Kaggle.

[10] D. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *ICLR*, 2015.

[11] M. Pielka et al., "Spoiler in a TextStack: How Much Can Transformers Help?," arXiv:2112.12913, 2021.

[12] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot," PLoS ONE, 2015.

[13] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, 1995.